

mapping_pathfinding

Kartierung, Lokalisierung und Pfadplanung für die Duckiebot City Challenge

Nico Sander

21. Juli 2026

1 Ausgangslage und Aufgabenstellung

1.1 Was gegeben ist

Die Stadt wird als Graph beschrieben und liegt als Dictionary vor: Knoten sind Kreuzungen, Kanten sind Straßen. Jede Kreuzung besitzt bis zu vier Ausfahrten, im Folgenden *Ports* genannt, die mit 1 bis 4 nummeriert sind. Die Nummerierung folgt einer festen Konvention: die Ports laufen zyklisch um die Kreuzung, sodass sich 1 und 3 gegenüberliegen und Port 2 immer rechts von Port 1 liegt. Fehlt ein Port, so hängt an dieser Seite keine Straße, womit sich Kreuzungen mit drei und mit vier Armen einheitlich beschreiben lassen.

Das Dictionary bildet jeden Knoten auf seine Ports ab und jeden Port auf das Paar aus Nachbarknoten und dessen Port. Diese Abbildung muss symmetrisch sein: führt Port 1 von A nach Port 1 von B, so führt Port 1 von B zurück nach Port 1 von A. Im Paket liegt die Stadt in `config/city.json`, wird beim Start auf genau diese Symmetrie geprüft und von allen Nodes aus derselben Datei gelesen, sodass es nie mehr als eine Definition der Stadt gibt.

<pre>"nodes": { "A": { "1": ["B", 1], "2": ["C", 2], "3": ["C", 1], "4": ["B", 2] }, "B": { "1": ["A", 1], "2": ["A", 4], "3": ["C", 4] }, "C": { ... } }</pre>	Teststrecke: A ist die Kreuzung mit vier Armen, B und C sind T-Kreuzungen. Eine Strasse wird ueber ihre beiden Enden benannt, etwa A1__B1. Der Schluessel ist unabhængig von der Fahrtrichtung: von beiden Seiten ergibt sich derselbe. Ein Tor gilt damit automatisch fuer die ganze Strasse.
---	--

Aus der Portkonvention folgt die gesamte Abbiege-logik als reine Tabelle: für einen Bot, der über Port p einfährt, liegt Port $p + 1$ rechts, Port $p - 1$ links und Port $p + 2$ geradeaus. Da keine dieser Tabellen einen Port auf sich selbst abbildet, sind Wendemanöver nicht bloß verboten, sondern strukturell gar nicht ausdrückbar.

Über einzelnen Straßen hängen *Tore* in Form von AprilTags. Welches Tor über welcher Straße hängt, ist nicht gegeben, sondern genau das, was der Bot selbst herausfinden muss. Vor Ort angesagt wird lediglich die Reihenfolge, in der die Tore später durchfahren werden sollen, als Liste von Tag IDs.

1.2 Zu lösendes Problem

Der Bot muss die Aufgabe in zwei Phasen bewältigen:

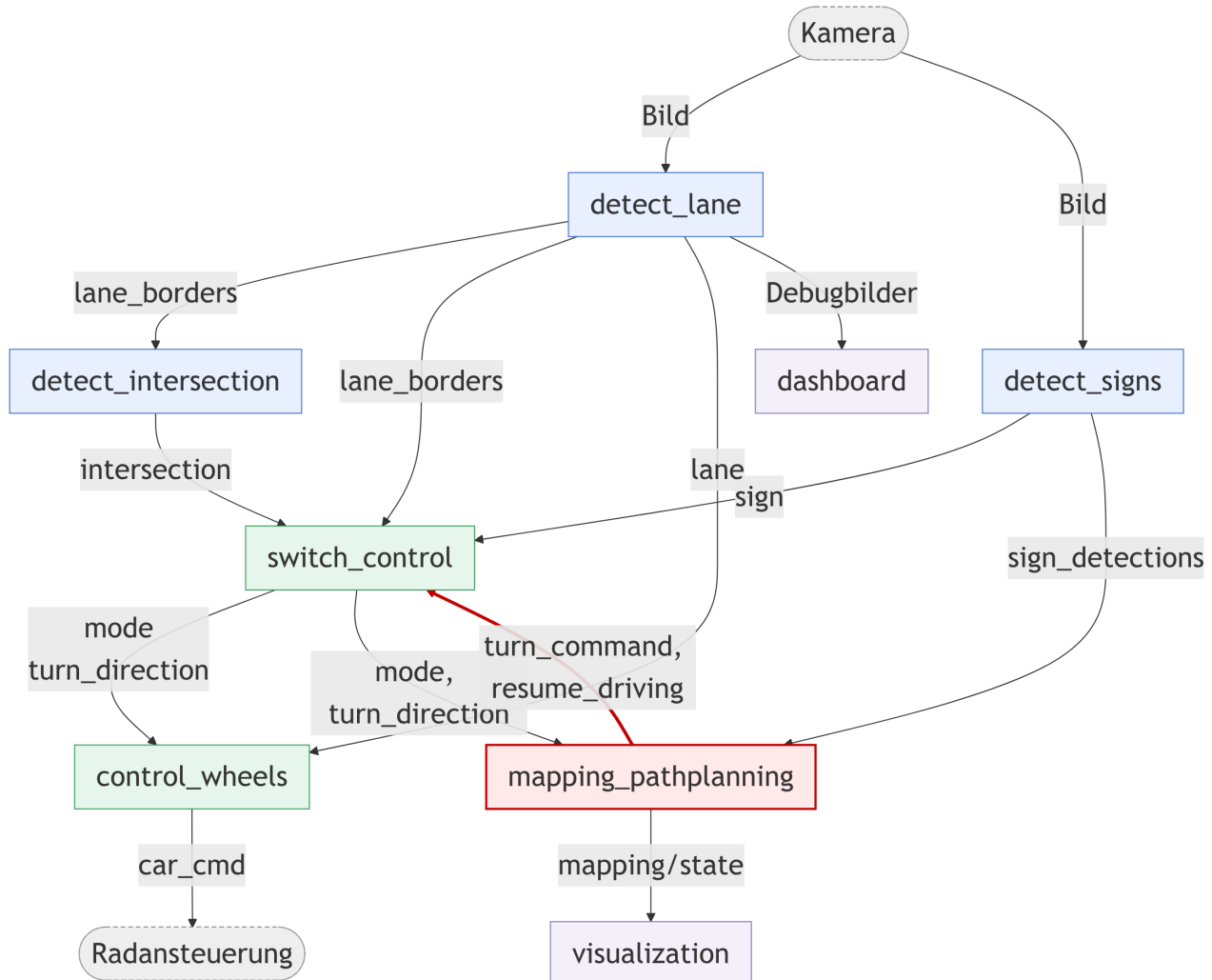
Kartierung (ungestoppt) Der Bot wird an einer bekannten Straße abgesetzt und muss jede Straße der Stadt mindestens einmal befahren, denn nur so kann kein Tor übersehen werden. Während der Fahrt ordnet er jedes erkannte Tor der Straße zu, über der es hängt, und misst nebenbei, wie lange er für jede einzelne Straße und für jedes Abbiegemanöver tatsächlich braucht. Anschließend hält er an und wartet.

Torlauf (gestoppt) Die Torreihenfolge wird angesagt, der Bot wird an eine bekannte Straße gestellt und muss die Tore in dieser Reihenfolge durchfahren. Gewertet wird die benötigte Zeit, weshalb die Route nicht nur korrekt, sondern möglichst schnell sein soll.

Daraus ergeben sich vier Teilprobleme, die das Paket lösen muss: dem Straßenverlauf zuverlässig folgen und Kreuzungen sauber queren; jederzeit wissen, wo im Graphen sich der Bot befindet; Tore korrekt der richtigen Straße zuordnen, auch wenn die Kamera über die Kreuzung hinweg in benachbarte Straßen blickt; und schließlich eine Route planen, die ohne Wendemanöver auskommt und die gemessenen Fahrzeiten berücksichtigt.

2 Systemüberblick und Kernkomponenten

Das System ist in drei Schichten aufgebaut. Die *Wahrnehmung* wertet das Kamerabild aus und liefert daraus Spurlage, Kreuzungszustand und erkannte AprilTags. Das *Verhalten* entscheidet daraus, was der Bot gerade tut, und setzt es in Radkommandos um. Die *Mission* verfolgt die Position im Graphen, baut die Karte auf, plant die Route und gibt der Verhaltensschicht vor, wohin an der nächsten Kreuzung abgelenkt wird. Damit ist der Regelkreis geschlossen: die oberste Schicht wirkt auf die mittlere zurück, statt nur mitzulesen.



Der ROS Node Graph, auf den Hauptdatenfluss reduziert. Kästen sind Nodes, Pfeile sind Topics, gestrichelt sind Kamera und Radansteuerung als externe Beteiligte. Die Farbe zeigt die Schicht: blau die Wahrnehmung, grün das Verhalten, rot die Mission, violett die Anzeige. Rot hervorgehoben ist die Kante von der Mission zurück auf das Verhalten (`/plan/turn_command` und `/plan/resume_driving`), die den Regelkreis schließt.

2.1 Wahrnehmung

detect_lane.py Segmentiert das Kamerabild mit einem U-Net in weiße Fahrbahnmarkierungen, gelbe Mittellinie und rote Haltelinien. Daraus berechnet der Node den Querabstandsfehler, also wie weit der Bot aus der Spurmitte steht, und veröffentlicht ihn auf `/detect/lane`. Zusätzlich publiziert er die geometrischen Spurgrenzen samt Position einer roten Linie im Bild auf `/detect/lane_borders` sowie Bilder zur Fehlersuche. Die gesamte Wahrnehmung läuft mit 30 Hz auf dem Laptop, etwa 20 ms pro Bild bei einem Budget von 33 ms.

detect_signs.py Erkennt AprilTags mit `pupil_apriltags` und publiziert auf zwei Topics, weil die beiden Abnehmer Unterschiedliches brauchen. `/detect/sign` enthält nur die ID des nächstgelegenen Tags und ist ungefiltert, denn Kreuzungsschilder müssen in jeder Größe erkannt werden. `/detect/sign_detections` enthält dagegen *jede* Detektion mit Eckpunkten, Fläche in Pixeln und einer Entscheidung über Annahme oder Verwerfen. Die Fläche braucht die Missionsschicht, um zu unterscheiden, ob ein Tor über der aktuellen Straße hängt oder erst hinter der Kreuzung.

detect_intersection.py Ein kleiner Logiknode, der aus der Spurgeometrie einen Kreuzungszustand ableitet: keine Kreuzung, Kreuzung nähert sich, Haltelinie erreicht. Das geschieht allein über die vertikale Position der erkannten roten Linie, ohne das Bild erneut anzufassen.

2.2 Verhalten und Aktorik

switch_control.py Der Verhaltensautomat und das Bindeglied zwischen Wahrnehmung und Mission. Er hält genau einen Zustand, den aktuellen Fahrmodus, und veröffentlicht ihn zusammen mit der Abbiegerichtung auf `/switch/mode` und `/switch/turn_direction`. Es gibt genau vier Fahrmodi, die im Normalbetrieb im Kreis durchlaufen werden: Spur folgen, Annäherung an eine Kreuzung, Halt an der Haltelinie und Queren der Kreuzung. Einen eigenen Wartezustand gibt es nicht; das Halten am Ende der Kartierung ist derselbe Modus STOPPED, nur festgehalten durch das Kommando TURN_COMMAND_HALT (siehe Abschnitt 3.2). Die Übergänge kommen aus der Wahrnehmung, etwa wenn die rote Linie nah genug ist oder die Haltelinie erreicht wurde.

Zwei Eigenschaften sind wichtig. Erstens bezieht der Node die Abbiegerichtung im Normalfall vom Missionsnode über `/plan/turn_command`; ist dieses Kommando veraltet oder fehlt es ganz, wählt er die Richtung ersatzweise anhand des Kreuzungsschilds. Ein ausgefallener Missionsnode führt so zu eingeschränktem, aber nicht zu gar keinem Fahrverhalten. Zweitens ist das Queren geschlossen geregelt: das Manöver endet, sobald wieder Fahrbahnmarkierungen sichtbar sind, und nicht wenn ein Timer abgelaufen ist. Die konfigurierte Dauer wirkt nur als Notausstieg. Genau das macht den Stack tolerant dagegen, dass der Bot nie exakt rechtwinklig an der Haltelinie steht. Nach Abschluss des Querens meldet der Node den vollzogenen Zustandswechsel, worauf die Missionsschicht ihre Position um eine Straße weiterrückt.

control_wheels.py Der einzige Node, der mit den Motoren spricht. Er übersetzt den aktuellen Fahrmodus in Radkommandos: beim Spurfolgen läuft ein Regler vom Typ PID auf dem Querabstandsfehler, dessen D Anteil tiefpassgefiltert ist, damit er bei 30 Hz stabil bleibt. Während einer Querung fährt er einen festen Bogen für die kommandierte Richtung, bis **switch_control** das Manöver beendet. Die gemessene Latenz von der Bildaufnahme bis zum Kommando beträgt rund 75 ms.

2.3 Mission, Kartierung und Planung

mapping_pathplanning.py Der Node, dem die Mission gehört, und die größte Einzelkomponente des Pakets. Er erledigt vier Dinge. Erstens die Lokalisierung, umgesetzt als Koppelnavigation auf dem Graphen: die geglaubte Position rückt um eine Straße weiter, sobald **switch_control** eine abgeschlossene Querung meldet. Zweitens die Kartierung, also das Eintragen der Tore auf die Straßen, über denen sie gesehen wurden. Drittens die Planung: nach jeder Querung wird die gesamte Restroute neu geplant, was bei dieser Stadtgröße billig ist und dazu führt, dass eine korrigierte Position sofort eine korrigierte Route ergibt. Viertens das Kommandieren, also das Publizieren der nächsten Abbiegerichtung auf `/plan/turn_command`. Zusätzlich veröffentlicht er den vollständigen Missionszustand, bestehend aus Karte, Toren, Pfad, Position und Phase, als JSON auf `/mapping/state` und bietet einen Service `start_gate_run` für den Phasenwechsel an.

planner.py (ohne ROS) Dijkstra über gerichtete Zustände aus Knoten und Einfahrtport, also über die Aussage „ich nähere mich dieser Kreuzung und bin über diesen Port eingefahren“. Über Knoten allein lässt sich nicht planen, denn wohin der Bot als Nächstes darf, hängt davon ab, wie er angekommen ist. Der Zustand identifiziert zugleich die gerade befahrene Straße, was es erst möglich macht, einzelne Straßen und Tore gezielt anzusteuern. Kanten sind mit einer geschätzten Dauer gewichtet, wobei ein Zug aus Halt, Abbiegen und Straße besteht, sodass die Suche wirklich schnellere und nicht bloß kürzere Routen bevorzugt; eine Rechtskurve ist deutlich billiger als eine Linkskurve. In der Kartierungsphase treibt dasselbe Modul eine gierige Erkundung, die stets die nächstgelegene noch nicht befahrene Straße ansteuert.

city_map.py (ohne ROS) Der Graph selbst: Portgeometrie, Laden und Validieren der Stadtdefinition sowie **GraphMap**, der laufende Kartenzustand mit der Information, welche Straßen befahren wurden und welches Tor über welcher Straße hängt.

gate_detection.py (ohne ROS) Entscheidet, welchen Torsichtungen geglaubt werden darf. Ein Tor wird dauerhaft auf eine Straße geschrieben, der erste Eintrag gewinnt also. Das ist Absicht: die Kamera sieht über die Kreuzung hinaus, spätere Sichtungen stammen daher aus größerer Entfernung und schlechterem Winkel. Vor diesem unwiderruflichen Schreibvorgang stehen drei Filter. Im Stand und während einer Querung wird gar nicht kartiert. Die Fläche des Tags muss über einem Schwellwert liegen, der aus aufgezeichneten Fahrten kalibriert wurde. Und dasselbe Tag muss über mehrere aufeinanderfolgende Bilder auf derselben Straße gesehen werden.

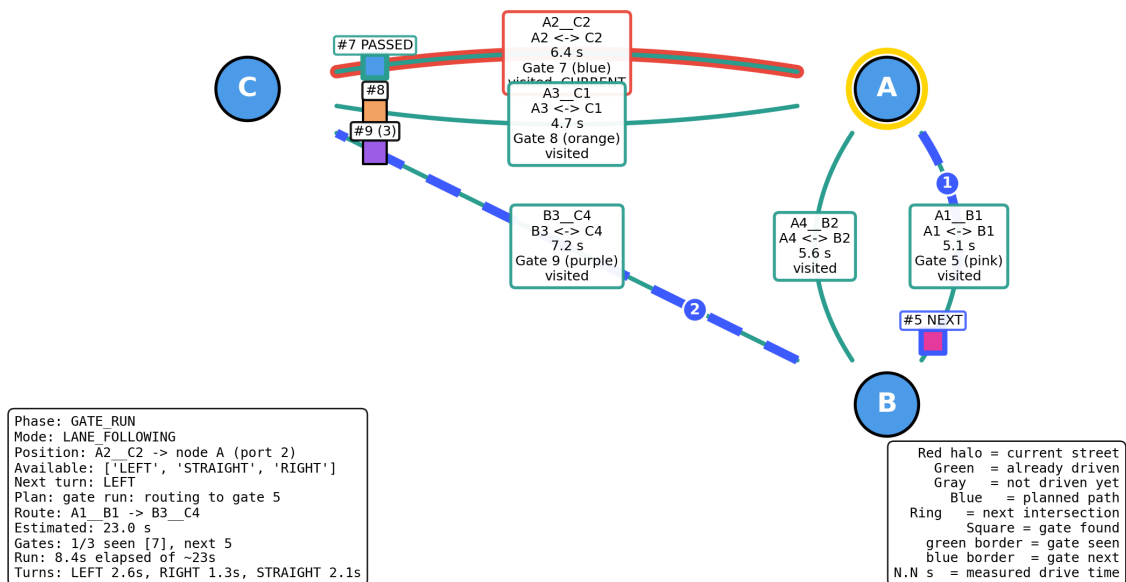
run_timing.py (ohne ROS) Misst, wie lange jede Straße und jedes Abbiegemanöver tatsächlich dauert. Als gemeinsame Zeitbasis dienen die von **switch_control** publizierten Fahrmoduswechsel, sodass sich die gemessenen Teile lückenlos und überlappungsfrei zu einem vollen Zug zusammensetzen. Die Kartierung ist ungestoppt, dort zu messen kostet also nichts, und der gestoppte Lauf wird anschließend gegen reale Zahlen statt gegen eine pauschale Schätzung geplant. Mehrfachmessungen einer Straße werden über den Median zusammengefasst.

crossing.py und graph_layout.py (ohne ROS) Zwei kleine Hilfsmodule. Das erste entscheidet, wann eine Kreuzungsquerung beendet ist, nämlich sobald nach einer minimalen Blindzeit wieder Markierungen sichtbar sind. Das zweite berechnet die Zeichengeometrie der Karte, mit der getesteten Eigenschaft, dass sich gezeichnete Straßen außerhalb eines Knotens niemals kreuzen dürfen.

2.4 Bedienung und Visualisierung

visualization.py Ein Fenster, das laufend die drei von der Aufgabenstellung geforderten Dinge sichtbar macht: die aufgebaute Karte mit den gefundenen Toren, den geplanten Pfad und die geglaubte Position des Bots.

Duckiebot Mapping Graph -- GATE_RUN



Das Visualisierungsfenster während des Torlaufs. Grün sind bereits befahrene Straßen, rot umrandet die aktuelle, blau gestrichelt der geplante Pfad mit Schrittnummern, der Ring markiert die nächste Kreuzung. Die Quadrate an den Straßen sind die gefundenen Tore. Die Ansicht ist offline erzeugt (**tests/render_example_visualization.py**) und zeigt einen konstruierten, aber vom echten Planer gerechneten Zustand, keinen Mitschnitt einer Fahrt.

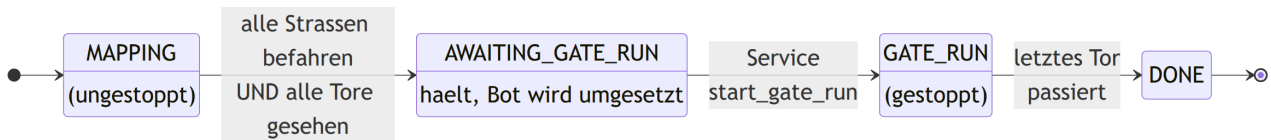
dashboard.py Ein Fenster zur Fehlersuche in der Wahrnehmung, das Rohbild, Segmentierungsmasken und die Rahmen der erkannten AprilTags samt gemessener Fläche in einer Ansicht kachelt. Es ist dasselbe Dashboard, das bereits in Challenge 1 und Challenge 2 verwendet und dort beschrieben wurde; hinzugekommen ist lediglich die Anzeige der Tagflächen, die für die Torzuordnung gebraucht wird.

mission_tui.py und mission_setup.py Ein menügeführtes Frontend für die gesamte Session. Der strukturell wichtige Punkt ist, dass *ein* **roslaunch** über beide Phasen hinweg am Leben bleibt, denn Karte, Tore und gemessene Zeiten liegen im Missionsnode und ein Neustart zwischen den Phasen würde sie verworfen. Die Bedienoberfläche besitzt daher den Launchprozess und wechselt die Phase über Parameter und den Serviceaufruf. **mission_setup.py** validiert die eingegebenen Antworten, vor allem die Startposition, die vor dem Start gegen den Stadtgraphen geprüft wird, und baut daraus das Startkommando zusammen.

3 Zustandsdiagramme und Ablaufdiagramme

3.1 Missionsphasen

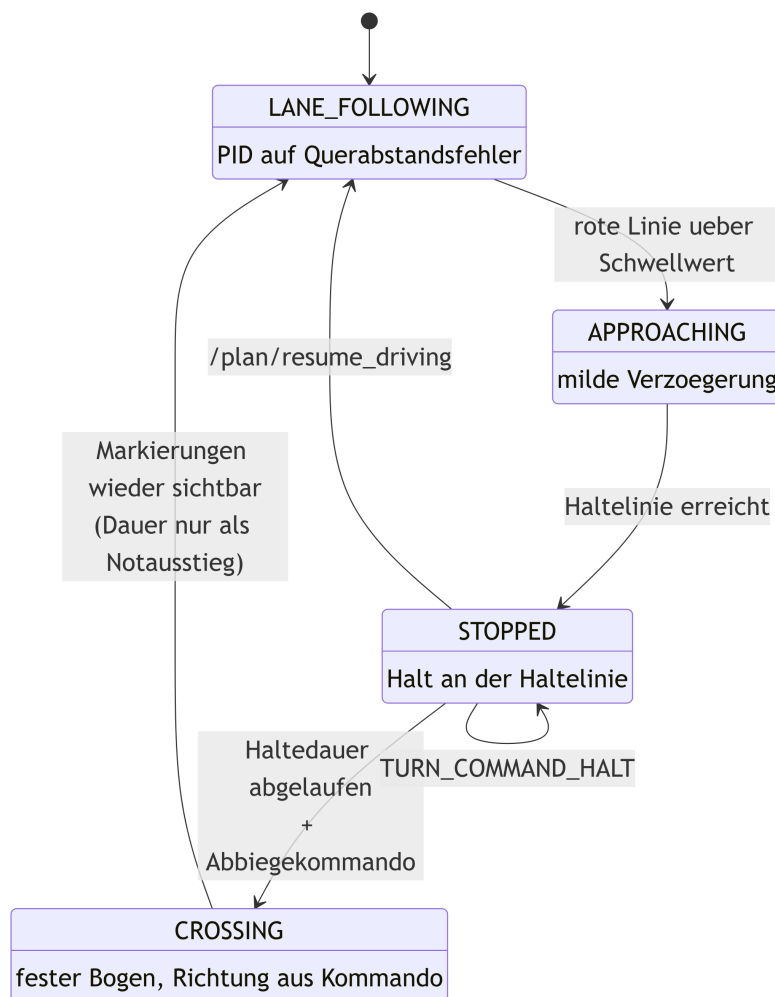
Der übergeordnete Automat von `mapping_pathplanning.py` kennt vier Phasen: die beiden Fahrphasen, das Warten dazwischen und den Abschluss. Der Übergang aus der Kartierung verlangt ausdrücklich beides: alle Straßen befahren *und* alle Tore gesehen. Eine befahrene Straße allein ist noch kein gesehenes Tor, und ein Torlauf gegen eine unvollständige Karte würde ins Leere planen.



Zustandsdiagramm der Missionsphasen, benannt wie die Werte von `MissionPhase`. Zwischen Kartierung und Torlauf liegt mit `AWAITING_GATE_RUN` eine eigene Phase: der Bot hält an der Haltelinie und rührt sich nicht, bis er von Hand an die Startstraße des Torlaufs gestellt und der Service aufgerufen wurde.

3.2 Verhaltensautomat

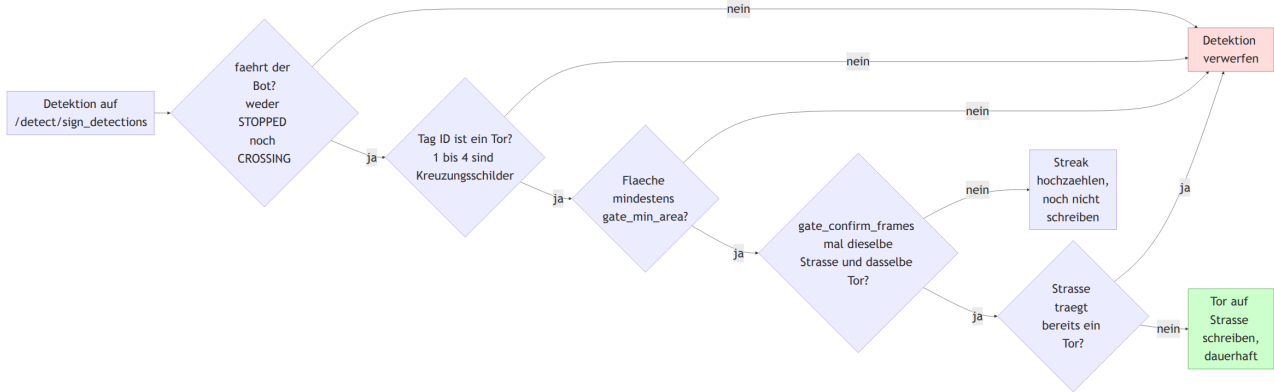
Der Fahrmodusautomat aus `switch_control.py` wird im Normalbetrieb im Kreis durchlaufen. Interessant sind die beiden Kanten, die aus diesem Kreis ausbrechen: `TURN_COMMAND_HALT` hält den Bot in `STOPPED` fest, statt ihn queren zu lassen, und `/plan/resume_driving` holt ihn von dort zurück ins Spurfolgen, ohne dass er ein Querungsmanöver ausführt. Vor einem Bot, der gerade an den Anfang einer Straße getragen wurde, liegt keine Kreuzung.



Verhaltensautomat von `switch_control`, benannt wie die Werte von `DriveMode`. Die Schleife an `STOPPED` ist kein eigener Modus, sondern das Festhalten am Ende der Kartierung. Dieselben Zustandswechsel sind zugleich die Zeitbasis von `run_timing.py`: als Abbiegezeit zählt `STOPPED` bis `CROSSING` bis zurück nach `LANE_FOLLOWING`, als Straßenzeit der Abschnitt von dort bis zum nächsten `STOPPED`. Beide stammen aus derselben Quelle und setzen sich deshalb lückenlos zu einem vollen Zug zusammen.

3.3 Torzuordnung

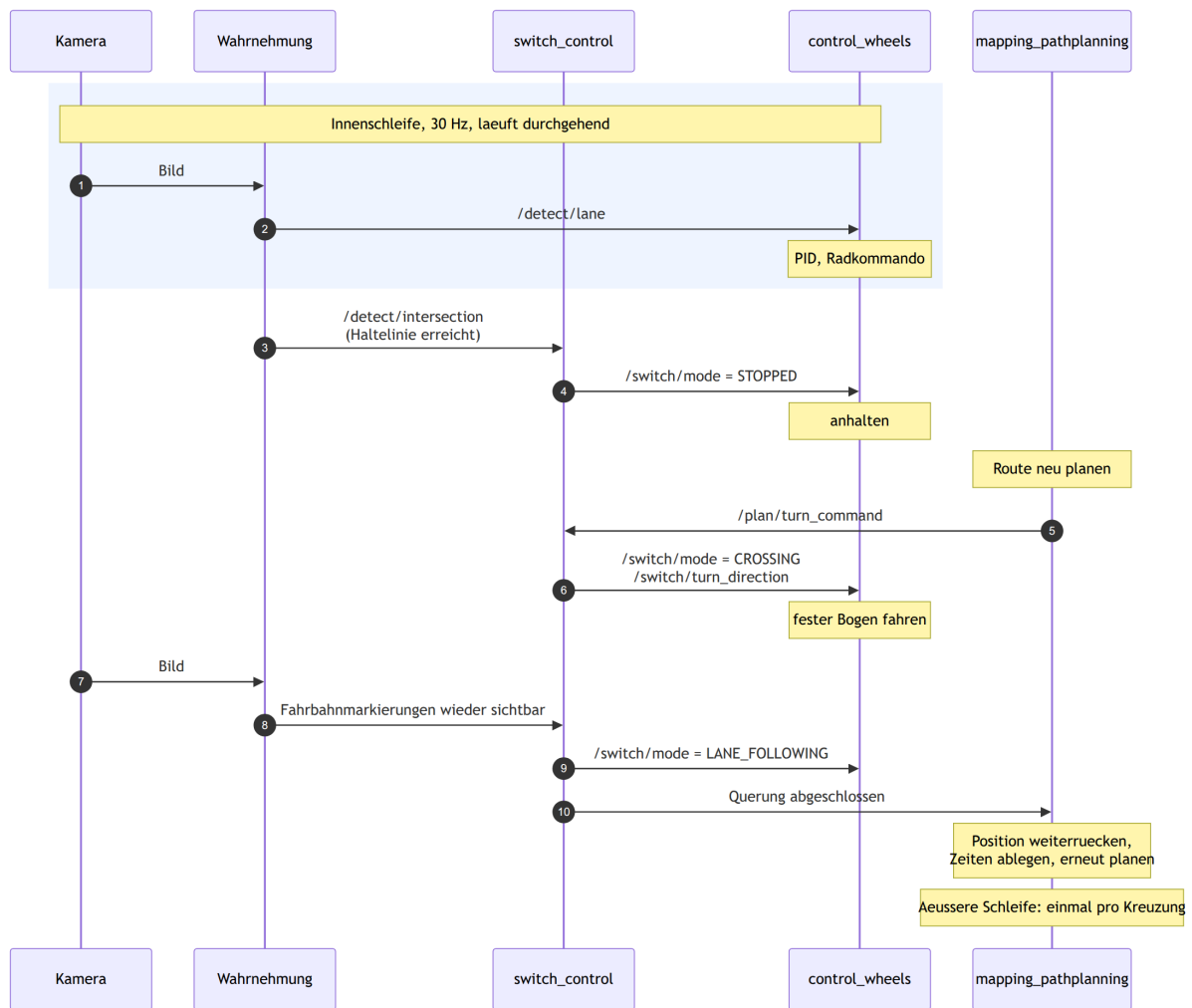
Der folgende Ablauf entscheidet für jede einzelne Detektion, ob sie auf die Karte geschrieben wird. Da der erste Eintrag auf einer Straße endgültig ist, steht vor dem Schreibvorgang bewusst eine ganze Kette von Filtern.



Ablaufdiagramm der Torzuordnung für eine Detektion aus `/detect/sign_detections`. Alle Ablehnungen laufen in denselben roten Kasten, grün ist der einzige Zweig, der schreibt. Die Gründe dafür unterscheiden sich: im Stand und beim Queren wird gar nicht kartiert; die IDs 1 bis 4 sind Kreuzungsschilder und keine Tore; eine zu kleine Fläche heißt, dass das Tag hinter der Kreuzung steht, denn die Fläche fällt mit dem Quadrat der Entfernung; und eine Straße, die bereits ein Tor trägt, wird nie überschrieben, weil spätere Sichtungen von weiter weg und aus schlechterem Winkel stammen. Die Bestätigungsserie fängt zusätzlich den Fall ab, dass ein einzelnes Bild fehldetektiert.

3.4 Regelkreis über eine Kreuzung

Zuletzt der zeitliche Ablauf an einer Kreuzung, an dem sich die beiden ineinanderliegenden Schleifen des Systems ablesen lassen: die schnelle Innenschleife der Spurregelung mit 30 Hz und die äußere Schleife, die genau einmal pro Kreuzung durchlaufen wird.



Sequenzdiagramm über eine Kreuzung. Die Spur *Wahrnehmung* fasst `detect_lane`, `detect_signs` und `detect_intersection` zusammen. Der blau hinterlegte Bereich ist die durchgehend laufende Innenschleife, die gelben Kästen sind interne Arbeit des jeweiligen Nodes. Schritt 5 ist die Stelle, an der die Mission auf das Verhalten zurückwirkt, Schritt 10 die Rückmeldung, mit der die Mission ihre Position eine Straße weiterrückt.

4 Konfiguration und Tests

Alle abgestimmten Konstanten liegen in `config/config.json`, darunter Zeiten, Reglerparameter, Schwellwerte und das Kostenmodell des Planers. Die Karte ist `config/city.json`, die rein kosmetische Zuordnung von Tor ID zu Farbe steht in `config/gates.json`. Argumente beim Start überschreiben ausgewählte Werte für einen einzelnen Lauf.

Da der Entscheidungskern ohne ROS auskommt, wird er offline verifiziert: Portgeometrie und die Invariante, dass kein Wendemanöver möglich ist, Validierung der Stadt, die Filter für Torsichtungen, Erreichbarkeit im Planer aus jedem Startzustand, alle Torreihenfolgen aus allen Startzuständen, geringe Abdeckung, vollständige Missionssimulationen und synthetische Gitterstädte. Zusätzlich betreibt `launch/simulate.launch` den echten Missionsnode gegen einen simulierten Fahrer und testet damit die tatsächliche ROS Schicht mit Topics, Timern und Phasenwechsel, ohne Kamera, Motoren oder Roboter.